Uppsala University



Independent Project in Information Engineering

1DT350

Design and Implementation of an Automated Record-to-Invoice System via API Integration

Authors:

David Ebbersten
Anton Hedlund
Daniel Rosenfeld
August Wannfors
Klas Widebeck

June 3, 2026

Abstract

Small and medium-sized companies often suffer from administrative burdens and data discrepancies in their billing cycles, where a ripple effect is created, scaling manual labor with the company growth. The implemented system breaks this link and ensures the company can grow without the manual workload increasing. To automate the previously manual processes for PT Hemma, an automated administrative service was developed. It is built using a Next.js framework with the frontend being developed using React and the backend connected to a relational database.

The system integrates with the Fortnox API to transform record data from JSON objects into invoices, addressing the gap between internal records and the financial administration. To ensure data integrity and reduce discrepancies a verification flow was established, requiring manual approval by the consultants before API transmission. The gathered results show that this automated approach significantly reduces manual administrative labor while minimizing the risk of data discrepancies. Administrator workload is reduced from approximately eight hours down to one to two hours.

Sammanfattning

Små och medelstora företag möter ofta administrativa utmaningar i form av datadiskrepanser i sina faktureringscykler, där mängden manuellt arbete ökar i takt med företagets tillväxt. Det implementerade systemet bryter denna effekt och försäkrar att företaget kan växa utan att manuellt arbete ökar. För att automatisera de tidigare manuella processerna utvecklades en administrativ tjänst. Det är byggt med ramverket Next.js där frontend utvecklades med React och backend kopplades till en relationsdatabas.

Systemet integreras med ett Fortnox API för att omvandla journaldata från JSON objekt till fakturor, vilket minskar klyftan mellan interna daganteckningar och den ekonomiska administrationen. För att säkerställa dataintegritet och minska diskrepanser etablerades ett verifieringsflöde som kräver manuellt godkännande före API-överföring. De insamlade resultaten visar att detta automatiserade tillvägagångssätt minskar den manuella administrativa bördan samtidigt som risken för datadiskrepanser minimeras. Administratörens arbetsbörda minskades från ungefär åtta timmar ned till en till två timmar.

Contents

1 Introduction **4**

1.1 Background and context . . . 4
1.2 Problem description . . . 5
1.3 Purpose . . . 6
1.4 Research questions . . . 6
1.5 Related work . . . 6

2 Method **7**

2.1 Technical framework . . . 8
2.2 Invoice creation . . . 8
2.3 System evaluation and user testing . . . 8

3 System architecture **9**

3.1 Data storage . . . 10
3.2 Referential integrity . . . 10
3.3 Data flow . . . 11
3.4 Role-based access control (RBAC) . . . 11
3.5 Data aggregation . . . 12

4 Materials **13**

4.1 Legacy system . . . 13
4.2 API integration and external services . . . 13
4.3 Database framework . . . 13
4.4 Interface logic and client page . . . 14
4.5 Assisting tools . . . 14

5 Results **14**

5.1 System testing and labor efficiency . . . 14
5.2 Logical and technical validation of verification flow . . . 15
5.3 UI implementation . . . 15

6 Discussion **16**

6.1 Ethics and sustainability . . . 17
6.2 Limitations and analysis . . . 18
6.3 Possible future work . . . 18
6.4 UI analysis . . . 19
6.5 Contributions . . . 20
6.5.1 David Ebbersten . . . 20
6.5.2 Anton Hedlund . . . 20
6.5.3 Daniel Rosenfeld . . . 21
6.5.4 August Wannfors . . . 21
6.5.5 Klas Widebeck . . . 21

7 Conclusion **22**

1 Introduction

In many small and medium sized companies, administrative work such as invoicing may still remain a manual task. While fundamental business activities are digitalized, they still lack optimization of their fundamental features. Optimizing and improving efficiency in already established features and moving away from manual work leads to a smoother and more uninterrupted workflow [12]. Automation of repetitive administrative tasks can lead to an improved operational efficiency and reduce human error [5]. A risk of continuing to work in inefficient systems is as the business grows, so does the manual work load. This in turn increases the risk of human error and limits the company's scalability and ability to expand [1]. This problem is especially occurring in businesses where worked hours or performed work needs to be documented and no control of correct documentation is done as well as organizations that work with customers and employ an invoicing system rather than direct payment. In these cases, these tasks require manual intervention which is not only time consuming and ineffective, but also prone to inconsistencies.

This project is an optimization and efficiency improvement of one such system where it aims to explore and analyze in what way manual tasks can be automated, as well as how to support a more streamlined time management in the company using API integration tools. The project will also research how to best incentivize correct documentation in a record system.

1.1 Background and context

Record systems are information systems used to store and manage data related to business operations. Record systems often contain data such as activity logs, transaction records or user entries [2]. They mostly depend on manual input from multiple users, which can in turn lead to delayed or inconsistent input or incorrect input altogether. The system is therefore entirely dependent on the correct input, and since the data is user generated and often varies, it becomes hard to enforce. The lack of enforcement creates a garbage in garbage out effect where the utility of the entire database is compromised. This means that for the system, the information collected needs to be accurate and precise, otherwise the programs that build on the data will be unreliable [11]. If the record system also supports the creation of invoices i.e. a combined record and invoice system, any inconsistencies or errors in the system will therefore affect the generated invoices.

A record system that also supports the integration of invoice generation generally includes an invoice system. The system handles the creation of invoices and management of financial transactions to and from parties concerned. They are vital in financial record keeping to track payments for accounting and are entirely reliant on, as a record system, correct input. They are sometimes entirely separate from the record system or sometimes integrated with it to allow for faster transfer of data and to avoid mismatch between the two systems. [10]. Integration is particularly effective as it eliminates the need for double-entry, where data must be copied from one system to another, which is a process that is inherently prone to errors.

To generate invoices in a dual system where it handles both records and financial matters, API:s are needed. API stands for *application programming interface* and works essentially like a common language between two or more computer programs. Instead of a certain program needing to know how to do something from scratch, it can use the API to simply ask another one to trigger a certain progress or to ask for specific data. The API enforces what is possible, what can be requested and how the response should look like between two applications [3]. This is vital in generating invoices where a record system would have to be much more advanced to do what predefined invoicing systems can do in addition to what it already does.

The types of Fortnox invoices used by PT Hemma are the supplier invoice and the regular invoice. The supplier invoice is used when a company receives a bill from an external party, such as a consultant or supplier, for services rendered. In this context it allows consultants to bill PT Hemma for their work each month. The regular invoice works in the opposite direction, where the company itself bills an external party for services provided. In PT Hemma's case this means billing the end customers, i.e. the individuals purchasing personal training sessions, for the sessions they have attended. In the case of PT hemma the consultants also send regular invoices to the company for accounting purposes, these are referred to as self-invoices in this thesis.

1.2 Problem description

The current record system at PT Hemma relies on a high degree of manual intervention to transition from record data to financial output. Specifically, the process of generating and sending invoices to both consultants and customers. As stated earlier, it needed to be done manually by extracting data from internal records and reformat the data through an external software. This manual workflow creates an administrative stress on the system. Consequently, as the number of customers and consultants grow, the time spent billing them each month will also increase. This diverts time and resources away from away from more strategic business operations and creates a bottleneck.

This company inefficiency is further augmented by the human factor when entering data in the record system. In the existing system, the errors usually are only identified by the end of the billing month. These discrepancies in the documentation and record system have to be corrected by the administration, often involving direct communication between administration and consultants. This not only increases the total working hours required for financial administration but also heightens the risk of billing inaccuracies. Without a systematic way to validate data, the administrator must assume the role of a manual clearinghouse for the data, a role which becomes unstable as the company grows.

1.3 Purpose

The primary objective of this project is to design and implement an automated record-to-invoice system within an existing record system, using API integration. This is to address inefficiencies in the current administrative pipeline. It serves a dual purpose where it both aims to decrease the manual labor in the billing cycle of a company along with minimizing the amount of data discrepancies in the company database.

To achieve this reduction in manual labor, the record-to-invoice system automates the data extraction and compilation of consultant notes, transforming company data into structured invoices via the Fortnox API. To mitigate the data discrepancies between the central administration and the consultants, the system introduces an accountability shift. This is done by transferring the responsibility of data verification before invoice generation from the administration to the consultants themselves. A mandatory data verification is sent from the administrator to each consultant, where they must review and approve their own logged hours prior to invoice generation. This approach ensures that data errors are corrected, preventing administrative bottlenecks.

1.4 Research questions

- How can the implementation of an automated administrative service reduce manual labor in the billing cycle for an organization?
- In what way can a multi-step verification flow ensure that trainers and administrators agree on the billing data the system presents?

1.5 Related work

There are many record systems that try to minimize manual labor and support invoices management. One such system is Frenda that specializes in dental healthcare. Frenda aims to create an effective workflow and has implemented features such as online appointment booking and digital check-in for patients when arriving at the clinic, thereby minimizing the administrative workload. Frenda handles invoices by keeping track of patient balances and eligibility for social welfare benefits and then using integrations to other services such as Resurs Bank, PayZmart and Walley to generate and manage the invoices.

Another system is CTnotes, a record system designed for psychotherapists. Regarding invoices, CTnotes handles the creation of them in the system itself and uses an integration with Fortnox to automate bookkeeping processes. This reduces the need for manual financial administration. While these systems demonstrate effective ways to minimize the administrative workload, they primarily focus on client-facing billing. Neither Frenda nor CTnotes have support for self-invoicing to the workers.

A key point of the new system for PT-Hemma is the actual automation of invoice creation in order to minimize manual work as much as possible. There exist several big companies that give this service, one of which is Fortnox. Since Fortnox is already in use by the administration and has an API with extensive documentation it is a good choice of service.

Johannes Danielson and Kasper Lindberg have performed a similar project where a user study on the personnel of a company was made[4]. Since such a study was not possible in this project, this study possessed valuable data. They gathered that the system in use was also too time consuming due to, for instance, creating invoices. However, something that differs was the high level of complexity in their system causing the whole process to take additional time. This is not the case for this project's current system, but with careless implementation and design, the new system could suffer from high complexity, thus diminishing the amount of time saved on automation. This serves as a caution during development of the system that one should minimize complexity in systems not in use by technically skilled personnel as much as possible.

2 Method

The methodology of this project is divided into two primary areas to address the research questions systematically. Firstly a baseline level was established by observing the current manual labor required for the billing cycle and to measure the reduction in manual labor. This baseline was established through interviews and direct observation of the company's primary administrator interacting with the system, i.e. a qualitative assessment was to be conducted. Because historical logs of exact minutes spent were not available, the baseline relies on the administrator's operational estimates of working days prior to the project. This baseline was then compared to that of the new system.

To gain a clear understanding of the system design, system requirements were established throughout a series of structured consultations and interviews with the primary administrator. The objective of the design phase was to develop a system architecture that ensured data alignment and mutual consensus between the consultants and the administrator.

2.1 Technical framework

The implementation strategy focused on extending the existing repository to connect the record logs to an automated accounting service. The method relied on altering the database structure, converting raw data, and presenting this in a user friendly way. The given repository provided an architectural foundation to build upon, and the main method of implementing the automated billing system focused on extending this foundation. The aim of extending the database framework was to implement a way for the system to isolate and manipulate data sets from the company database, within a certain billing period. These data sets could then be sent to an external financial service to generate invoices. The success was measured by viewing that the time based logic successfully isolated the correct record notes, without including incorrect or overlapping data.

2.2 Invoice creation

To generate invoices, Fortnox was selected since it is currently employed by the company. The invoices for the consultants and for the clients are generated through the official Fortnox API. To ensure a secure integration, the method included using a sandbox development cycle with the Fortnox Developer Portal. This meant a dedicated test environment was established to allow for isolated testing of the API [9]. Through this sandbox environment, an evaluation of the JSON data exchange was carried out to ensure the database outputs successfully conformed to the external API's requirements. The system is designed to be connected directly to PT Hemma's live production Fortnox account for actual commercial use.

2.3 System evaluation and user testing

The final version of the system was evaluated and tested in regards to both hours of labor saved and how it enables the administrator and consultants to reach an agreement regarding billing data. The administrator were given tasks to perform without guidance where the work intended to be done by the consultants were done automatically in the backend . This was so that the admin could focus entirely on the functionality that concerned them. The evaluation of the test was done both by the observation of how effectively the admin could do their tasks, and by gathering the administrators personal experience of the workflow. They were also asked to estimate the new total time spent during the billing cycle.

The testing sequence required the administrator to complete the following tasks:

- **Navigate** to the admin dashboard
- **Verify** if an invoice basis has been sent to each consultant for a certain month
- **Generate** and send invoice bases for a certain month
- **Inspect** the details of the invoice basis
- **Attempt** to send consultant invoices
- **Observe** invoice basis acceptance status
- **Transmit** consultant invoices
- **Send** customer invoices

3 System architecture

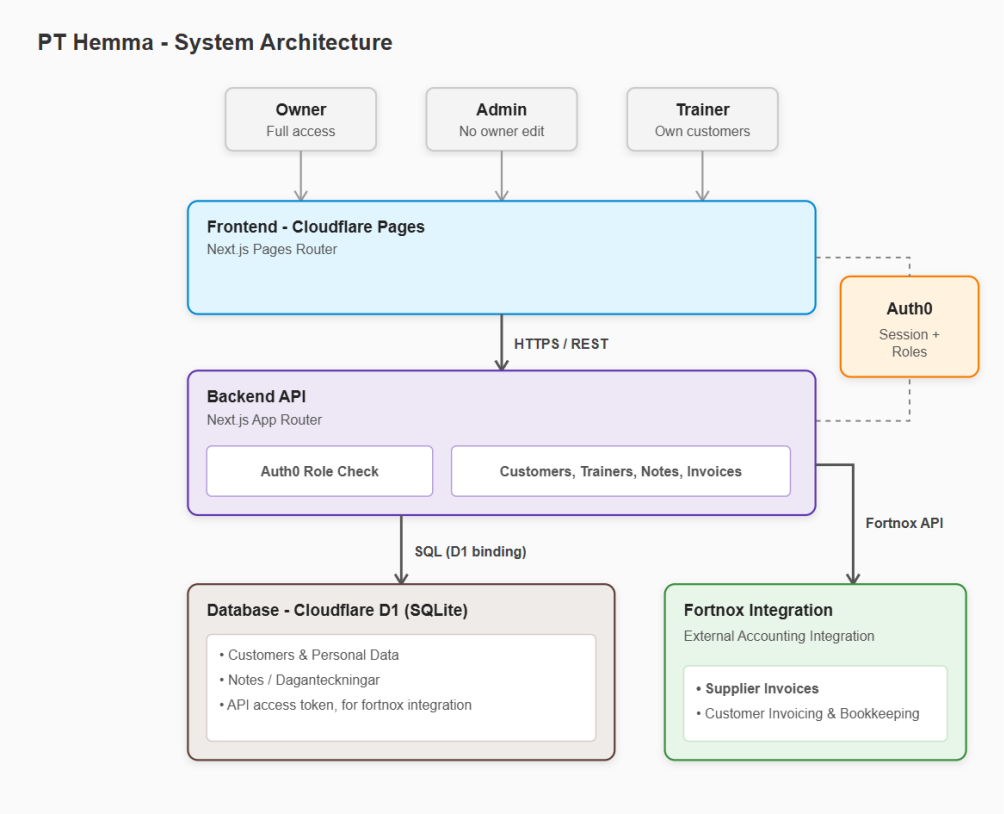


Figure 1: Overview of the system architecture

The architectural design shown in figure 1 largely inherits core components from the legacy system, with primary modifications centered around the integration of the external Fortnox API. This integration made it necessary to change the database schema to accommodate session notes and Fortnox authentication tokens as well as some changes in the old tables in the schema. Communication between the user interface and the serverless infrastructure is decoupled. The presentation layer interacts with the backend API by dispatching asynchronous HTTP requests over an encrypted HTTPS protocol [8], strictly adhering to RESTful architectural principles [7]. In the application code, this network communication is implemented using the native fetch() API within a TypeScript environment.

3.1 Data storage

The data in the record system is stored in a relational database and is split into several distinct entities. The four most important being clients, trainers, notes and tokens, with notes representing an entry by a trainer of a client. Individual notes do not depend on one another, they instead exist as independent records that directly reference specific clients and trainers through relational dependencies. By separating these entities rather than storing all client and session information inside a single table, the system follows the principles of database normalization. This means organizing attributes and relations of database objects, ensuring that the dependencies are properly enforced by database integrity constraints. Without normalization, if one changes the address of a client, and that address is stored in several different notes for example, it would mean changing the address as many times as it is stored. If normalization is applied the address exists in exactly one place instead and minimizes the risk of an update anomaly.

All information regarding the Fortnox tokens was stored in the database to ensure persistence between sessions and to prevent accessibility from the frontend website environment (which can occur when using client-side storage). The tokens are managed entirely from the backend via a function that enforces administrative authorization.

3.2 Referential integrity

To maintain data consistency and practice referential integrity, the system uses universally unique identifiers (UUID:s) as primary keys. In the case of the notes, referential integrity is enforced throughout the system with the use of foreign keys. The foreign keys create an association for a note with a trainer and a client, which thereby ensures that every session in the record system is tied to its real world counterparts, that being a client and a trainer. This in turn prevents the creation of orphaned data. I.e. data that is missing its dependencies. Enforcing these constraints at a database level rather than at the backend software code, a high level of program reliability is guaranteed. The software programs can thus operate under the assumption that every note in the system is a valid one which removes the need for constant validation checks.

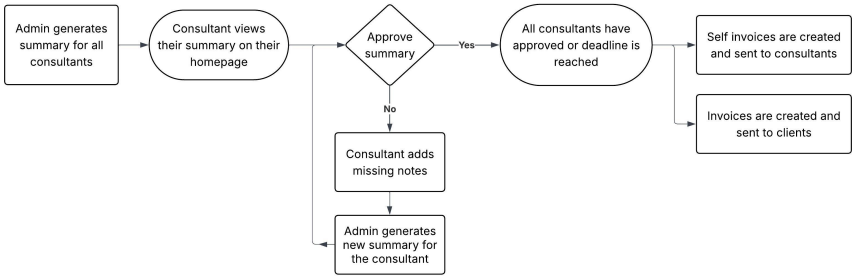


Figure 2: The system data flow

3.3 Data flow

In the system, each note carries essential information that regulates the invoices to be generated. They contain data about the specific client and trainer it regards but also at what date the session the note is based on occurred. When an invoice basis is to be generated, the system needs a way of knowing if a note already has been billed or not. The record system therefore uses time-windowed processing. This means that instead of using boolean flags that signal if a note has been billed or if it has been approved by an admin, it will use the timestamp tied to the note.

Time-windowed processing in the case of the record system means that a note's affiliation is not determined by the status of a certain attribute, but instead by a timestamp. Each note's date works as a primary sorting key. When the system is about to generate a basis for the invoices, a filtering process occurs in which the notes with the incorrect month are sorted out.

This in turns creates a distinction where data is aggregated at the request of the admin. After the invoice basis is created, consultants get the option of either accepting or refusing the basis. If they accept, the admin generates an actual invoice, and if they refuse the admin can send another one once details regarding the basis have been added by the consultant. Once all consultants have accepted their basis, the admin can create invoice bases for the clients.

3.4 Role-based access control (RBAC)

The combined record and invoice system enforces strict roles using something called role-based access control. This means access permissions are administratively associated with roles and users are made members with appropriate roles. This principle simplifies management in an organization while also providing flexibility in different security protocols [6]. In the system PT Hemma employs there are two distinct roles. Trainers and administrators. These roles have separate functions and varying levels of authority over the data.

The trainer role has permission to create, remove and read notes for the specific clients that the individual trainer is connected to. They are identified by a unique ID, the UUID, which holds control over their access. The admin role however possesses more permissions. They can create customers, trainers and notes as well as generate invoices for the entire database. Only the admin role can trigger the transition of notes from a raw state to an invoice basis which will be used to create the actual invoices. This contributes to a more secure system partly for the administrators and the sensitive data they hold, but also for customers who prefer their data being kept safely and in accordance with GDPR.

3.5 Data aggregation

The final stage of the flow of data is invoice basis generation. It's the part where the data is processed into a final product in which the trainer will accept or refuse the generated product. Aggregation refers to the process in which information from one or several databases are compiled for processing. The aggregation of notes in the journal system is the primary objective which this project aims to automate, since doing it manually means going through every note for every client that month.

To enable the aggregation of invoice bases containing all necessary information, additional fields were added to the note table in the database, including the ID of the trainer who created the note and a note type. The note types were introduced to identify which type of training session had been performed, which in turn determined both the amount charged to the client and the compensation paid to the trainer. An extra note type was also added for notes that should not be included in invoice bases, intended for cases where trainers want to add information about a client without registering a training session.

The system uses database-level functions to retrieve the relevant notes based on a trainer or client ID and a selected month, applying the time-windowed processing approach. The generated invoice basis is then presented in a web format which was developed as the frontend side of the invoice system, building on the existing record system.

4 Materials

The following section covers all the materials used. This includes the existing legacy system that this project builds upon, frameworks, tools, external services etc.

4.1 Legacy system

The legacy record system provided an existing base which had support for basic data logging but lacked automation. The record system was built using a modern web framework called Next.js which in turn is built on the JavaScript library React. Through this, a basic record system could be developed and implemented in a single stack repository. The repository follows a monorepo structure, meaning that both the client-facing user interface and the server-side backend logic reside within the same codebase. The system employs a hybrid routing approach using Next.js, where the App Router is responsible for all API routes and backend logic, while the Pages Router is used exclusively for frontend rendering. The system utilises Tailwind CSS as its styling framework. Tailwind CSS is a CSS framework, where styles are applied directly in the markup, HTML, through predefined utility classes rather than writing custom CSS.

4.2 API integration and external services

The integration with the external financial service relies on the Fortnox API. This API bridges the gap between raw data collected in daily notes and the service offered by Fortnox which packs this data into client and consultant ready invoices. The integration utilizes REST (Representational State Transfer) architecture meaning the record system can interact with Fortnox using standard HTTP requests [7] such as GET and POST. Along with the standard HTTP requests, the API also uses JSON (JavaScript Object Notation) as the primary data exchange format which aligns with how data is processed and packed from a Next.js application.

4.3 Database framework

Data persistence is handled by the Cloudflare D1 relational database which is built upon the SQLite engine. The database organizes data using SQL schema which works as the blueprint for how data regarding notes, trainers and clients is stored. The database schema enforces specific relational constraints and data types, ensuring that database anomalies are avoided. Additionally, enforcing these constraints supports the function of data aggregation allowing for the collection of data from different places. These collections can then be structured into the JSON format which the Fortnox API uses.

4.4 Interface logic and client page

The user-facing administrative dashboard is constructed using React functional components managed within Next.js. React acts as the coordinator of everything and is responsible for the verification flow being displayed properly to the administrator. React breaks down the interface into modules which were needed to build the administration dashboard, along with its integration with the Cloudflare D1 database, simplifying data fetching.

4.5 Assisting tools

The tools used during development were primarily git, Github, Wrangler, Trello and Discord. Since the company was already using Github, an existing repository was available to build upon. Wrangler was used to run and test the system locally during development. Trello was used for project management, including tracking tasks, organizing work, and monitoring ongoing progress. Discord was used for easy communication and sharing information and resources between the group members.

5 Results

This chapter evaluates the system across three core areas: labor efficiency and administrative workload shifts (Section 5.1), the logical and technical validation of the verification workflow (Section 5.2), and the user interface implementation (Section 5.3). Together, these assessments demonstrate the system's efficiency, reliability, and usability in relation to the research questions.

5.1 System testing and labor efficiency

To assess the potential effectiveness of the developed automated invoice system in reducing manual labor within the billing cycle, a qualitative assessment with the main administrator at PT Hemma was conducted, which included a structured test of the system. This assessment focused on the perceived workload before and after the implementation from the administrators perspective. It highlighted the shift from manual workflow to an automatic one along with the verification feature for invoice generation. The administrator was able to determine an approximate total time needed for one billing cycle in the new system. This was one to two hours. The test was also done in order to determine if the system could fulfill the requirements of reaching an agreement between admin and consultant. According to the administrator, the system workflow was clear and precise so that a legally binding contract could be written between the consultants and admin. This way they could reach an agreement with no gray areas left to interpretations.

5.2 Logical and technical validation of verification flow

The administrator from PT Hemma emphasized that the addition of an invoice basis for each consultant, where a consultant had to approve of their logged hours before payment is processed, would greatly reduce the risk of human errors in the system. Human errors such as missing entries or delayed entries altogether. The combination of this verification flow and the ensuing administrative overview was described as outstanding. The administrator also expressed that a final stage for consultants where they need to accept or refuse their invoice basis before the data is transmitted to the Fortnox API, provided a sense of security and control.

While the assessment with the admin provided a qualitative overview, the technical performance of the system was verified through internal tests. They were designed so that the materials in use, such as the Next.js framework, the React frontend and the Fortnox API for example, would be fully validated and interact correctly. The time-windowed processing was tested across several different ranges so one could observe it correctly isolated invoice bases for one month. This was successful and proved not only that the database and its schema functions correctly, but also that the written TypeScript programs interacted correctly with the developed frontend programs written in React and the central storage database.

Secondly, mock-invoices were generated through the Fortnox API from JSON objects to observe the performance of the API. These tests were also successful, establishing that the Fortnox implementation worked. Technical and qualitative tests of the automated billing system thereby indicated that the system can successfully isolate data until approved, which answers the second research question by demonstrating how the verification workflow guarantees data consistency between trainer and administrator.

5.3 UI implementation

The final user interface implemented mainly consists of two simplistic pages with a similar style to the rest of the website. Figure 3 shows the page created for the admin to send and view bases, as well as create invoices for both trainers and clients. Shown in figure 4 is the consultant view. It has the purpose of letting the consultant view a basis that an admin has sent. A sent basis could be approved or denied by the consultant which would then show on the admin page. Other UI changes to the website include support for connecting each customer and trainer to the Fortnox API to allow for invoice creation. This was done by adding a field for customer-numbers and supplier-numbers for customers and trainers respectively.

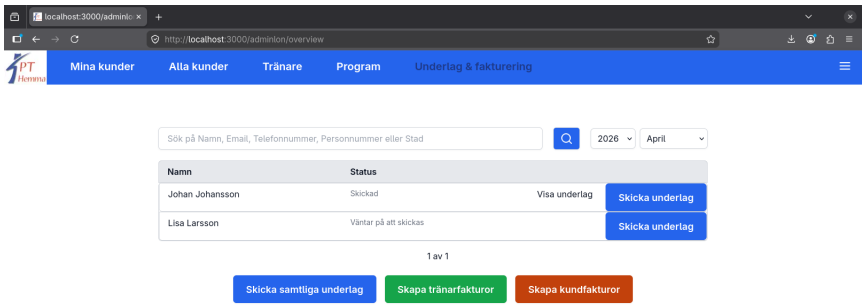


Figure 3: The admin billing view

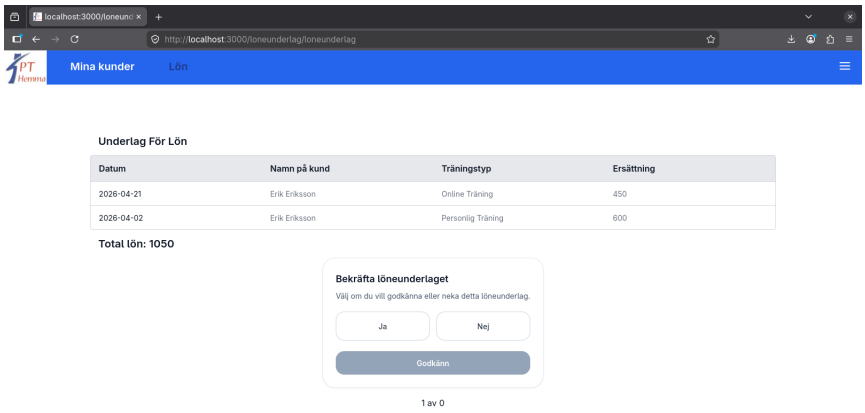


Figure 4: The admin billing view

6 Discussion

This chapter discusses the implications, constraints, and broader context of the developed billing system. It explores the ethical and environmental considerations of the implementation (Section 6.1), analyzes the system's operational limitations (Section 6.2), and outlines opportunities for future development (Section 6.3). Finally, it reviews the user interface (Section 6.4) and project contributions (Section 6.5) before concluding the study (Section 6.6).

As stated earlier, the legacy system functioned primarily as a digital note keeping service. The administrator was required to seek out consultants to ensure all notes were entered and cross-reference them with client lists to generate invoices. This manual process was earlier described as both tedious and time-consuming, requiring roughly eight to ten hours of active labor spread across two to three business days. Following the implementation of the automated invoice system the administrator observed a shift in the workload. Instead of the admin role previously functioning as a data extractor and coordinator, the role has now shifted to a data overseer position. Compared to the current manual workflow, this change reduced the time spent on monthly processing to about two hours, directly addressing the first research question by demonstrating how automation decouples administrative burden from company growth. The administrator could also determine that the workflow was clear enough to hold up in a legal contract between the admin and consultants. By signing a contract an agreement can be reached and solve the issue of ambiguity between the two parties.

6.1 Ethics and sustainability

When creating systems handling personal data, privacy, security and confidentiality are at the center of the ethical perspective. In the automated invoice system, the security is handled through role based access control which dictates who can view what in the system. This prevents consultants from viewing information about clients which they do not work with, and it prevents consultants from viewing information about each other. Along with this, the system is also designed so that it does not manipulate or distort client and/or consultant data, keeping the data intact and safe in the Cloudflare D1 database. To mitigate the risk of accidental data exposure to the client-side interface, the system implements a strict separation of concerns by isolating the user interface within a Next.js Pages Router environment from the backend logic managed via the App Router. Data access does not rely on client-side state, instead data retrieval and change (via HTTP GET and POST requests) are strictly done with the backend API [8]. This layer enforces authorization checks, validating the identity and role permissions of the requesting session. Consequently, arbitrary client-side requests are rejected. For example, if a user possessing only 'Trainer' privileges attempts to execute a customer creation routine, the backend blocks the unauthorized execution before it interacts with the database. Worth mentioning is that this hybrid routing system is inherited from the legacy system and is not by any means the only way of doing this.

From a sustainability standpoint, the automated invoice system does not directly reduce carbon emissions or impact large-scale energy grids. It does, however, touch upon environmental sustainability in the form of resource-efficient software design and digital waste reduction. In the legacy workflow, processing the billing cycle required manual data extractions and repeated database interactions spread across multiple days. By automating this process into an optimized pipeline, the total number of database operations, network requests, and active server queries is minimized. This strains the CPU less, making the new workflow more energy efficient, which demonstrates that efficient software system design can minimize its environmental impact.

6.2 Limitations and analysis

This project demonstrates that automation can substantially improve administrative workload in service-based organizations. By connecting the record system with the Fortnox API, the billing process went from a mostly manual process to a largely automated one. The administrator no longer needs to manually create invoices from looking at individual notes, but instead only supervises an automated process.

An important outcome was the multi-step verification flow of the billing process. Instead of having a fully automated system, generating invoices directly from the notes, the system requires the consultants to view the monthly billing data and approve it before the invoices are sent. This reduces the likelihood of discrepancies caused by missing or incorrect notes, and helps ensure that the billing process each month will be done by the determined end-date. This also shifts some of the responsibility from the admin to the consultants.

One limitation of this study is the lack of measurement of discrepancies in the workflow. While the admin reported a large improvement in efficiency, measuring how the new system affects the agreement of the billing data is harder to measure. Without a larger study involving the consultants, or historical data of discrepancies in the past, the results in this regard are not concretely measurable

A notable security limitation of the current implementation resides in the storage mechanism for the Fortnox API access and refresh tokens. Presently, these credentials are saved as plain text within the Cloudflare D1 database. While Cloudflare inherently provides infrastructure-level encryption at rest, this defense mechanism does not safeguard the data against application-layer vulnerabilities. Specifically, the tokens remain exposed to unauthorized exposure via compromised application endpoints or through users possessing elevated database access privileges. This should not pose any direct vulnerability issues but if you were to expand on the system in the future vulnerabilities could arise that expose the tokens. Although implementing application-level encryption (encrypting tokens prior to database insertion) would mitigate this risk by adding a robust layer of defense-in-depth, doing so was considered outside the immediate scope of this project.

6.3 Possible future work

Although the system has successfully automated large parts of the billing process, there are still things that could be implemented to improve efficiency and reduce mistakes further. For example, the system could automatically notify consultants and remind them to fill in notes and accept or decline whenever a payment basis is sent to them. This could speed up the process and make it more likely that consultants will remember to use the system.

The scalability would also need to be improved for the system to be used by very large companies. The system is yet to be tested for large datasets and high workloads. Currently there is a limit of 25 API calls per second which could be an issue for a larger scale company. A queue-based API handler could be implemented to deal with this. Furthermore, if time would have allowed, a UI testing phase with the consultants who will utilize the system would have been done. Currently, only a qualitative operation interview with the administrator was performed, who was tasked with navigating the system and performing certain tasks. To fully know if the system was user friendly and foolproof, several interviews could have been conducted with the consultants and tasking them with various challenges is the system.

A significant area for future work is conducting UI testing with the consultants who will be using the system. As mentioned in the discussion, the testing performed during this project was limited to the administrator's perspective, leaving the consultant-facing parts of the verification flow untested with actual users. Arranging a testing session where consultants are tasked with navigating the approval flow, reviewing their invoice basis and either accepting or rejecting it, would provide valuable insight into whether the interface is intuitive enough for non-technical users. Such testing could reveal usability issues that are not apparent from the creator or admins perspective, and the findings could be used to further refine the UI before the system is deployed for active use.

Another area for future improvement is the security of the stored Fortnox API tokens. Currently, the access token and refresh token are stored as plain text in the Cloudflare D1 database. While Cloudflare provides encryption at rest at the infrastructure level, this does not protect the tokens from being read by anyone with access to the database through the application itself, such as through a compromised endpoint or elevated user privileges. Implementing application-level encryption of the tokens before they are written to the database would add an additional layer of security, ensuring that even in the event of unauthorized database access the tokens could not be directly used to interact with the Fortnox API.

6.4 UI analysis

The newly developed user interface was intentionally designed to align with the visual style of the legacy system. Existing React components and Tailwind utility classes were reused where possible, and newly added elements were kept within the same color scheme to maintain visual consistency. The rationale behind this decision was to minimize the learning curve for both the administrator and the consultants, lowering the threshold for adopting the new features. A lower threshold for adoption is relevant to the second research question, as familiarity with the interface reduces the risk of user error or confusion during the verification and approval flow, which in turn supports agreement on billing data.

The testing conducted covered all use cases from the administrator's perspective, and the results indicate that the administrator was able to adapt to the new parts of the system without difficulty. However, no UI testing could be arranged with the consultants, which is a notable limitation. Since the consultants are central to the verification flow, being the ones who must review and approve the invoice basis, it is not possible to fully validate the usability of the system without their input. Arranging such testing was not feasible within the time constraints of the project, and would also have required additional coordination effort from the client, who would have needed to involve the consultants on behalf of the project. This is elaborated upon further in the future work section.

6.5 Contributions

6.5.1 David Ebbersten

Responsible for maintaining the project Trello board, ensuring that decisions made during group meetings were reflected in the board and that the workload remained organized throughout the project. The primary technical contribution was the Fortnox API integration. This began with researching the Fortnox API documentation and setting up the sandbox test environment in the Fortnox Developer Portal, which allowed for isolated testing before connecting to the live production account. This was followed by implementing a complete OAuth 2.0 authorization flow to obtain and store access tokens, refresh tokens, and token expiry times in the Cloudflare D1 database. A token refresh mechanism was also implemented to automatically renew the access token when expired, and once the authorization flow was stable, correctly structured API requests were built and sent to Fortnox to generate invoices. On the frontend, new UI components were styled to match the existing design, reusing the same color scheme and button styles. Green and red popup indicators were added to give the administrator clear visual feedback on whether an invoice transmission succeeded or failed. Additionally contributed to writing the thesis.

6.5.2 Anton Hedlund

Responsible for coordinating and leading several project meetings with the client regarding planning, workflow design and system requirements. Contributed to defining the overall workflow of the system by helping translate the client's needs into technical requirements. Also contributed to research, analysis and implementation of Fortnox API, with focus on supplier invoices. This was later followed by rigorous testing of the system in order to find bugs on vulnerabilities. The testing included using the system in unintended ways and finding security issues not implemented correctly. Additionally contributed to writing of the thesis.

6.5.3 Daniel Rosenfeld

Responsible for extending the system backend and ensuring reliable communication between the client, server, and database. The main contribution involved the aggregation and management of invoice bases, as well as implementing the required modifications to the database structure. The work included analyzing the existing backend built with Next.js API routing, identifying the information required for an invoice basis, and determining what additional data needed to be stored in the database tables. This was followed by implementing the logic for creating, handling and processing the invoice bases. Additionally contributed to the writing of the thesis.

6.5.4 August Wannfors

August was contribution to the project spanned both technical foundation design and structural project coordination. On the technical side his primary focus was data aggregation and database configuration. Because the project lacked access to live production data about customers, trainers and company notes due to privacy constraints, this data had to be created. The first challenge he solved was to correctly identify where these files should be created and which ones should be modified. August then successfully designed and executed the database seeding strategy, inserting the mock data into the Cloudflare D1 database. With this data, the team now had a sandbox to test database queries with. Furthermore, he was responsible for determining the correct way to extract this data and format it into structured JSON payloads.

Beyond the code and the technical implementation, August acted as the primary point of contact with the client, by managing all communication, scheduling the team's meetings, and running collaborative "workshops" to figure out what the system requirements needed to be. Finally he was also responsible for the thesis writing and academic documentation. August took the lead on drafting and revising the thesis drafts based on supervisor and client feedback and structuring the chapters, as well as designing the visual presentation materials.

6.5.5 Klas Widebeck

Mainly responsible for extending the system frontend, implementing UI features and logic for the admin page. This includes styling the page to fit in with the rest of the website and make the design as simple as possible. This was done by keeping the color scheme in accordance with the rest of the system and adding helpful popups and confirmation windows in an attempt to make the system fool-proof. The frontend work also included communication with the backend, creating, fetching and displaying data. Also added support for customer-numbers and supplier-numbers to allow for connecting customers and trainers to individuals in the Fortnox system, which is used in the invoice creation. Furthermore, work included extensive testing and bug-fixing as well as writing on the thesis.

7 Conclusion

This project set out to automate the billing process at the company PT Hemma to reduce manual labor and ensure agreement between consultants and employer through a multi-step verification flow. An automated system for the billing process was built upon the already existing record system. The record system uses Next.js and the Cloudflare D1 database, with the newly upgraded system using integration with the Fortnox API for invoice generation.

The results show that the implementation of the automated invoice creation has greatly reduced manual administrative labor. Before the automated invoice system was implemented, the process took about two to three days each month, where the administrator had to manually create invoices based on information written throughout the record system. After implementation, the labor has successfully been reduced to routine system inputs and awaiting consultant approval, taking one to two hours instead of two to three days.

The verification flow where the consultants approve the suggested invoice basis should theoretically incentivize the consultants to write correct notes in the record system since the notes now directly correlates to the consultants pay. Unfortunately there was neither enough time nor a clear way to perform a measurable study on how incentivized the consultants were by the created invoice system.

References

[1] David H. Autor. Why are there still so many jobs? the history and future of workplace automation. *Journal of Economic Perspectives*, 29(3):3–30, 2015.

[2] Helen L. Chatfield. The development of record systems. *The American Archivist*, 13(3):259–267, 1950.

[3] B. Cooksey. *An Introduction to APIs*. Zapier, Inc., 2014. Accessed: 2026-05-07.

[4] J. Danielson and K. Lindberg. Business process automation for a production company. Degree project, Malmö University, Faculty of Technology and Society, Malmö, Sweden, 2025. Accessed: 2026-04-27.

[5] Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. *Fundamentals of Business Process Management*. Springer Publishing, 2 edition, 2018.

[6] David F. Ferraiolo, Janet A. Cugini, and D. Richard Kuhn. Role-based access control (rbac): Features and motivations. Technical Report Annual Computer Security Applications Conference, National Institute of Standards and Technology (NIST), 1995.

[7] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Ph.d. dissertation, University of California, Irvine, 2000.

[8] JJ Geewax, Anshuman Mitra, and Michael Wood. *Mastering API Architecture: Design, Operate, and Evolve API-Based Systems*. O'Reilly Media, Inc., Sebastopol, CA, USA, 2024.

[9] F. Hedman. En undersökning av hur Fortnox API kan användas för att automatisera bokföring. Degree project, University of Gävle, Department of Computer Science, Gävle, Sweden, 2017. Accessed: 2026-04-27.

[10] Intuit QuickBooks. Invoice guide: Definition and examples, 2024. Accessed: 2026-04-09.

[11] Monique F. Kilkenny and Kerin M. Robinson. Data quality: "garbage in – garbage out". *Health Information Management Journal*, 47(3):103–105, 2018.

[12] McKinsey Global Institute. A future that works: Automation, employment, and productivity. Research report, McKinsey & Company, 2017.